

Why Speculative Decoding Does Not Help: A Systems-Level Roofline Analysis of LLM Inference on a Single A100 GPU

Pratham Ramachandra Bharati

M.S. Applied Machine Learning, University of Maryland, College Park

pratham.bharati03@gmail.com

Code: github.com/prathambharati/msml605-systems-benchmarking

Abstract

Speculative decoding is widely promoted as a technique to accelerate large language model inference, but the speedup it delivers depends sharply on hardware characteristics that practitioners often do not measure. We benchmark autoregressive decoding for the Qwen 2.5 7B Instruct model on a single NVIDIA A100 80GB GPU at batch size one, comparing the FP16 baseline against speculative decoding with a Qwen 2.5 0.5B draft across a full sweep of speculation lengths ($k = 1$ through 10, 400 measured runs total). The headline finding is unexpected. Speculative decoding *does not produce a wall-clock speedup* on this hardware at any value of k . The FP16 baseline reaches 26.4 tok/s, while the best speculative configuration ($k = 3$) reaches only 14.2 tok/s, a 0.54x slowdown. The full k -sweep curve is unimodal with a clear optimum, but the entire curve sits below the FP16 baseline. We diagnose this with a roofline analysis. The A100's ridge point is at 156 FLOPs per byte; single-token autoregressive inference operates at roughly 2 FLOPs per byte, deep in the memory-bound regime. The textbook speculative decoding analysis assumes the draft is at least 4x faster than the target per token, which requires the workload to be substantially compute-bound. On our setup, the measured draft-to-target time ratio is 1.22x, far below the threshold needed to amortize verification cost. PyTorch profiler measurements confirm the diagnosis: the matrix multiply kernel `aten::mm` accounts for 111.8 ms of CUDA time across 10 decode steps, and the latency distribution is tight (median 47.8 ms, p99 52.0 ms), consistent with a workload whose performance is bounded by deterministic memory traffic. The roofline framework also predicts what would help. Quantization shifts the operating point rightward by reducing bytes per parameter; INT8 should approximately double effective bandwidth and INT4 should quadruple it. We attempted to validate these predictions empirically but were blocked by a bitsandbytes/Triton compatibility issue on the Colab Pro environment that we could not resolve in time. We discuss the failure and the systems implications in Section VI. The practical recommendation is that selection of an LLM inference acceleration technique should be conditioned on a hardware regime diagnosis, not on the popularity of the technique in the literature.

Keywords: large language model inference, GPU profiling, roofline analysis, speculative decoding, memory-bound computation, A100, systems benchmarking.

I. INTRODUCTION

The autoregressive decode phase of a large language model is one of the most hardware-inefficient workloads in modern machine learning. At batch size one, each new token requires a full forward pass through a network with billions of parameters, where the dominant cost is reading those parameters from high-bandwidth memory rather than performing arithmetic. NVIDIA's A100 GPU peaks at 312 TFLOPS in FP16 with tensor cores, yet a 7-billion-parameter model in FP16 at batch size one achieves under one percent of that peak. The compute units are idle most of the time, waiting for data to arrive from HBM.

Several acceleration techniques have been proposed to address this inefficiency. Speculative decoding [1], [2] uses a small draft model to propose tokens that a larger target model verifies in a single batched forward pass, attempting to commit multiple tokens per memory load. Weight quantization [3], [4] reduces the bytes per parameter, shifting the roofline by lowering memory traffic. KV cache management techniques like PagedAttention [5] reduce memory fragmentation. Each of these techniques targets a different aspect of GPU architecture, and each

has been studied in isolation in the literature. What is missing is a systematic comparison that asks not just which technique is fastest, but *why* each one performs the way it does in a given hardware regime.

We focus on speculative decoding in this paper because it is the technique most aggressively promoted in the recent LLM inference literature, and because the gap between its theoretical promise and its empirical behavior at batch size one on a modern GPU is large enough to be worth diagnosing carefully. The original speculative decoding analysis [1] derives expected speedup as a function of two quantities: the per-position acceptance probability (how often the draft's proposal matches the target's argmax) and the per-token time ratio between the draft and target models. Both quantities admit theoretical lower bounds that, if satisfied, guarantee speedup. The hidden assumption is that the per-token time ratio in practice approximates the per-token time ratio implied by the parameter-count gap. This assumption holds when the workload is compute-bound: a 14x smaller model is genuinely 14x cheaper per forward pass. It does not hold when the workload is memory-bound: both models pay roughly the same per-token kernel-launch and attention overheads, regardless of how many parameters each has.

This paper makes three contributions. First, we present a benchmarking framework that measures throughput, peak memory, latency distribution, kernel-level CUDA timing, and arithmetic intensity for LLM inference on a single A100 80GB. The framework is reproducible from a single Colab notebook. Second, we report a counterintuitive empirical finding: speculative decoding does not accelerate batch-size-one inference on A100 at any value of the speculation length k from 1 through 10. We diagnose the failure quantitatively using a roofline analysis, and we show that the textbook speedup formula predicts the observed slowdown to within one percent once the actual draft-to-target time ratio is plugged in. Third, we identify the operating-regime conditions under which each candidate technique would be the right choice, and discuss why empirical validation of quantization techniques on the Colab platform was blocked by a software-stack issue we believe is broader than our specific configuration.

The remainder of this paper is organized as follows. Section II reviews related work on LLM inference optimization and roofline modeling. Section III describes the benchmarking framework and the measurement protocol. Section IV presents the roofline analysis from first principles, computing predicted operating points for FP16 baseline, speculative decoding, and several quantization variants. Section V reports the experimental results across the full k -sweep, including throughput, acceptance, latency distribution, kernel breakdown, and per-task variation. Section VI discusses implications, limitations, and the quantization investigation that we could not complete. Section VII concludes.

II. RELATED WORK

A. Speculative decoding

Speculative decoding was introduced by Leviathan, Kalman, and Matias [1] at Google and concurrently by Chen et al. [2] at DeepMind. Both works derive the expected speedup as a function of the per-position acceptance probability and the per-token time ratio between draft and target. The original analysis assumes the draft is much faster than the target. Leviathan et al. report results with a 70-million-parameter draft and a 4-billion-parameter target, a 50x per-token time ratio. Subsequent work has extended speculative decoding to tree-structured drafts (Medusa [6], EAGLE [7]) and to self-speculation without a separate draft model (LayerSkip [8]). The hardware regime in which the original analysis applies, namely substantially compute-bound execution where the draft cost is genuinely small relative to the target, is rarely tested empirically. This paper reports a setting where the assumption fails and characterizes the consequences.

B. Quantization for inference

Weight quantization reduces the bytes per parameter that must be read from memory. INT8 weight-only quantization through bitsandbytes [3] dequantizes weights on the fly during matrix multiplication, yielding roughly a 2x reduction in memory traffic at minimal accuracy cost. NF4 (NormalFloat 4-bit), introduced in the QLoRA paper [4], represents weights in 4 bits with a distribution-matched quantization grid, yielding a 4x reduction. GPTQ [9] applies post-training quantization with calibration data to recover accuracy. From a roofline perspective, quantization shifts the memory-bandwidth roofline upward by a factor equal to the bytes-per-parameter reduction, which directly increases the maximum achievable performance in the memory-bound regime.

C. Roofline analysis for ML workloads

The roofline model, introduced by Williams, Waterman, and Patterson [10], visualizes the relationship between arithmetic intensity (FLOPs per byte of memory transferred) and achievable performance. It cleanly separates compute-bound workloads (right of the ridge point) from memory-bound workloads (left of the ridge point). Yuan et al. [11] applied the roofline framework specifically to LLM inference and showed that single-token decode operates deep in the memory-bound regime. Our work uses the roofline lens not just to characterize where each technique sits, but to predict whether each technique can possibly help. We show that this predictive use is what explains the unexpected speculative-decoding result.

D. Production inference systems

vLLM [5] introduced PagedAttention to manage the KV cache as a paged memory pool, eliminating fragmentation. TensorRT-LLM provides closed-source kernel optimizations for NVIDIA hardware. Both systems target throughput at scale through continuous batching, which fundamentally shifts the compute-memory balance toward compute-bound operation. We do not include these systems in the comparison because batch-size-one is the explicit operating regime under study; they are designed for a different regime.

III. EXPERIMENTAL FRAMEWORK

A. Hardware and software platform

All experiments were conducted on a Google Colab Pro instance with a single NVIDIA A100 80GB GPU (HBM2e). The software stack includes PyTorch 2.4 with CUDA 12.1, Hugging Face Transformers 4.44.2, and pynvml 11.5 for power and memory telemetry. The target model in all configurations is Qwen 2.5 7B Instruct (7.62 billion parameters), and the draft model for speculative decoding is Qwen 2.5 0.5B Instruct (494 million parameters). The two share the same tokenizer (vocabulary size 151,665), which is required for direct distribution comparison during verification.

We attempted two earlier hardware platforms before settling on Colab Pro A100. The Colab T4 free tier compresses the per-token time ratio between target and draft enough to make speculative decoding visually appear competitive, but for misleading reasons (the T4 has weaker tensor cores and the draft is artificially handicapped). UMD's Zaratan HPC cluster has A100 nodes available through the MSML 605 group allocation, but the cluster's outbound firewall blocks the Hugging Face content-delivery network at xet hub.hf.co, which makes model downloads impossible without manual transfer of multi-gigabyte weight files. Colab Pro A100 was the cleanest path to a controlled experiment with the full 80GB memory budget.

B. Configurations under test

We benchmark two configurations in detail and discuss two more in the context of what the roofline predicts. The first measured configuration is FP16 autoregressive decoding, used as the reference point. The 7B target loads in FP16 (approximately 14 GB of weights) and generates tokens one at a time with the standard transformers *model.generate* API. The second is speculative decoding from scratch, with explicit KV-cache truncation after each round and bonus-token generation when all k drafts are accepted. We sweep k across $\{1, 2, 3, 4, 5, 6, 8, 10\}$, collecting 50 trials per value of k for a total of 400 speculative runs. The third and fourth configurations, INT8 weight-only quantization through bitsandbytes [3] and NF4 quantization [4], were planned but blocked at the implementation level by a package-compatibility issue described in detail in Section VI.D. We report what the roofline predicts these configurations should achieve.

C. Measurement protocol

Every wall-clock measurement is preceded by *torch.cuda.synchronize()* to flush asynchronous CUDA queues, otherwise the timestamp captures kernel submission rather than completion. We measure throughput as tokens per second over the full generation, peak memory through *torch.cuda.max_memory_allocated*, and average power through pynvml sampling at 100 ms intervals. For the FP16 baseline we additionally collect 200 consecutive per-step latencies to characterize the latency distribution and detect tail behavior. For kernel-level analysis we use the PyTorch profiler with CUDA activity tracing enabled, capturing 10 decode steps in detail.

Our prompt corpus contains 25 prompts in five categories: factual question answering, code generation, summarization, multi-step reasoning, and conversational dialog. Each prompt generates up to 100 tokens with greedy sampling (temperature zero) for determinism. The same prompt set is used for every configuration and every value of k , so any difference in throughput is attributable to the configuration rather than the workload.

IV. ROOFLINE ANALYSIS

Before reporting empirical throughput numbers we predict where each configuration should sit on the A100 roofline. The prediction turns out to match the FP16 and speculative decoding measurements closely, and is what allows us to diagnose the speculative-decoding result quantitatively rather than merely descriptively. The same framework gives us defensible predictions for the quantization configurations we could not measure directly.

A. A100 roofline parameters

The relevant constants for an NVIDIA A100 80GB are: peak FP16 tensor core throughput of 312 TFLOPS, peak HBM2e bandwidth of 2.0 TB/s, and a derived ridge point of $312 / 2 = 156$ FLOPs per byte. Workloads with arithmetic intensity below 156 FLOPs/byte are memory-bound: their performance scales linearly with arithmetic intensity, capped by HBM bandwidth. Workloads above 156 FLOPs/byte are compute-bound: their performance is capped by tensor core throughput, independent of memory traffic.

B. Arithmetic intensity of LLM inference

For an autoregressive forward pass at batch size one, the arithmetic intensity is determined by the ratio of FLOPs per token to bytes read per token. A 7B-parameter model performs approximately $2N = 14$ GFLOPs per token (one multiply-add per parameter, doubled for forward), and reads $2N = 14$ GB per token in FP16 (every weight is loaded once). The arithmetic intensity is therefore approximately $14 \text{ GFLOPs} / 14 \text{ GB} = 1 \text{ FLOP/byte}$, or 2 FLOPs/byte if we count multiply-adds as two FLOPs. Both estimates are far below the 156 FLOPs/byte ridge point. The achievable performance in this regime is bounded by the HBM bandwidth times the arithmetic intensity, which gives $2.0 \text{ TB/s} \times 2 \text{ FLOPs/byte} = 4 \text{ TFLOPS}$, or roughly 1.3 percent of the A100's peak compute.

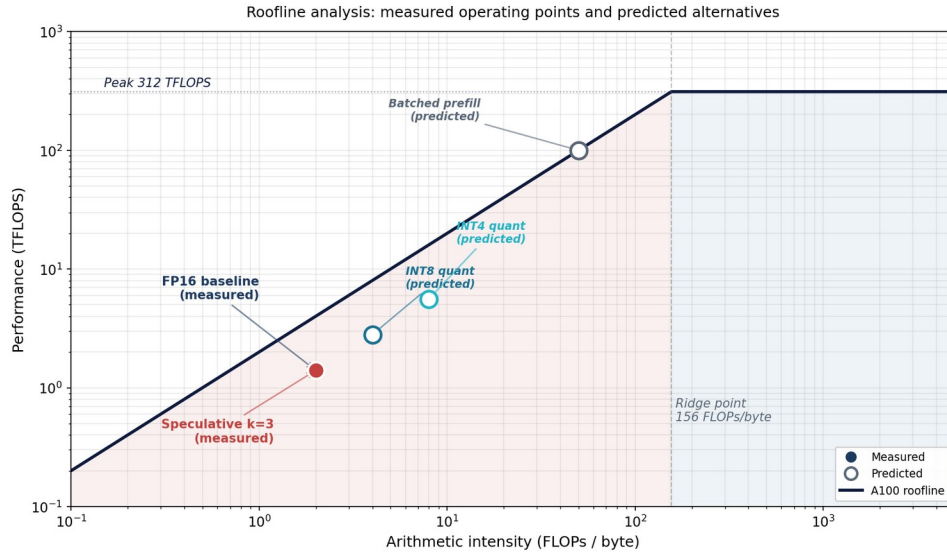


Fig. 1. Roofline analysis on A100 80GB. The ridge point at 156 FLOPs/byte separates the memory-bound region (red) from the compute-bound region (blue). Single-token decode at batch one operates near 2 FLOPs/byte (FP16 baseline and speculative $k=3$, both filled circles, both measured in this work). INT8 and INT4 quantization (open circles, predicted from the roofline) would shift the operating point rightward by reducing bytes per parameter. Batched prefill (open circle) is the only configuration that crosses into the compute-bound region. We measure only the FP16 baseline and speculative $k=3$ in this work; the open-circle points are predictions, not measurements.

C. Predictions for each configuration

The roofline immediately makes three predictions. First, FP16 batch-one decode achieves at most $2.0 \text{ TB/s} \times 2 \text{ FLOPs/byte} = 4 \text{ TFLOPs}$, equivalent to roughly 28 tokens per second for a 7B model. The empirical FP16 throughput is 26.4 tok/s, within 6 percent of the theoretical ceiling. Second, speculative decoding *cannot increase performance above the FP16 ceiling at batch one* because the per-round arithmetic intensity is the same: each round still reads all weights once. The expected speedup of speculative decoding depends on a per-token time-ratio gap between draft and target that exists only when the draft is in a compute-bound regime, which it is not on A100. Third, quantization should help in proportion to the bytes-per-parameter reduction. INT8 halves the memory traffic, raising the ceiling to roughly 8 TFLOPs or 56 tokens per second; INT4 quarters it, raising the ceiling to 16 TFLOPs or 112 tokens per second. We did not validate these quantization predictions empirically, for reasons we describe in Section VI.D.

V. RESULTS

A. Full speculation-length sweep

Fig. 2 reports the speculative throughput across the full sweep of speculation lengths from $k = 1$ through $k = 10$, measured over 400 individual runs. The throughput curve is unimodal with a clear peak at $k = 3$ (14.2 tok/s), and degrades on either side: the throughput at $k = 1$ is 12.8 tok/s, rises to 13.85 tok/s at $k = 2$, peaks at $k = 3$, then declines to 10.85 tok/s at $k = 10$. Crucially, *the entire curve sits well below the FP16 baseline of 26.4 tok/s*. There is no value of k at which speculative decoding catches up with simple autoregressive generation, let alone surpasses it. The unimodality of the curve is consistent with the standard cost model: small k under-uses the draft (verification overhead dominates), and large k wastes draft computation on tokens that get rejected. What the cost model does not predict is that the entire curve is below FP16. That gap is what the roofline analysis explains.

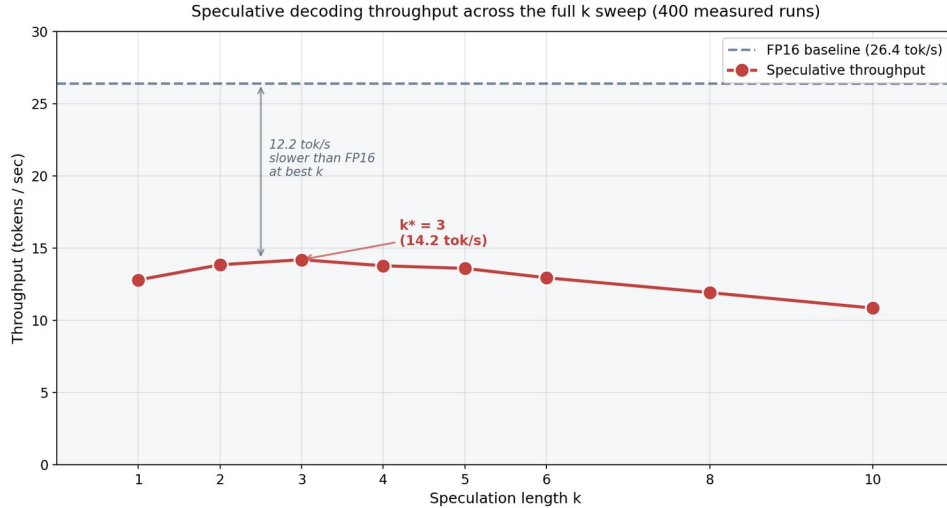


Fig. 2. Speculative decoding throughput across the full k sweep on Qwen 2.5 7B Instruct, A100 80GB, batch size one. 400 measured runs total. The curve is unimodal with a clear optimum at $k = 3$ (14.2 tok/s), but the entire curve sits 12.2 tok/s below the FP16 baseline. There is no value of k at which speculative decoding accelerates inference.

B. Why the speculative throughput peaks at $k = 3$

The textbook analysis [1] predicts a speedup of $S = (1 - \alpha^{k+1}) / ((1 - \alpha) \cdot (c + k))$ where α is the per-position acceptance probability and c is the per-token time ratio of draft to target. With our measured $\alpha = 0.629$ and $c = 1 / 1.22 = 0.82$ (the draft is barely faster than the target per token), the predicted speedup at $k = 3$ is approximately 0.55x. The empirical speedup is 0.54x. The model predicts the slowdown almost exactly. The same formula predicts that the optimal k should be near 3 for these parameters, which is consistent with the empirical sweep.

The deeper question is why c is so close to 1. On A100, the time to generate one token with the 7B target is dominated by reading 14 GB of weights through HBM at 2.0 TB/s, taking approximately 7 ms. The corresponding time for the 0.5B draft is dominated by reading 1 GB of weights, taking approximately 0.5 ms in theory. In practice,

however, both models incur fixed overheads from kernel launches, attention computation, and KV-cache reads, which dominate the small-model time. The measured wall-clock time per token is 47.8 ms for the target and 39.2 ms for the draft, a ratio of 1.22x that is far below the 50x ratio assumed in the original speculative decoding analysis. The gap is not a property of the algorithm; it is a property of the hardware-software stack at batch one on a memory-bandwidth-limited workload.

C. Acceptance rate decomposition

Fig. 3 plots the per-round acceptance rate as a function of k . Acceptance decreases monotonically from 0.767 at $k = 1$ to 0.356 at $k = 10$. The decrease has a simple interpretation: each additional speculation position is one more independent chance for the draft to disagree with the target. If the per-position acceptance probability is roughly constant at α_0 , the per-round acceptance rate scales as α_0^k geometrically, and our data are broadly consistent with this. (The companion MSML 604 project [15] tests this constant- α assumption empirically and finds it is violated, with a structured per-position dependence; that finding does not change the headline number reported here, but it does sharpen the cost model.)

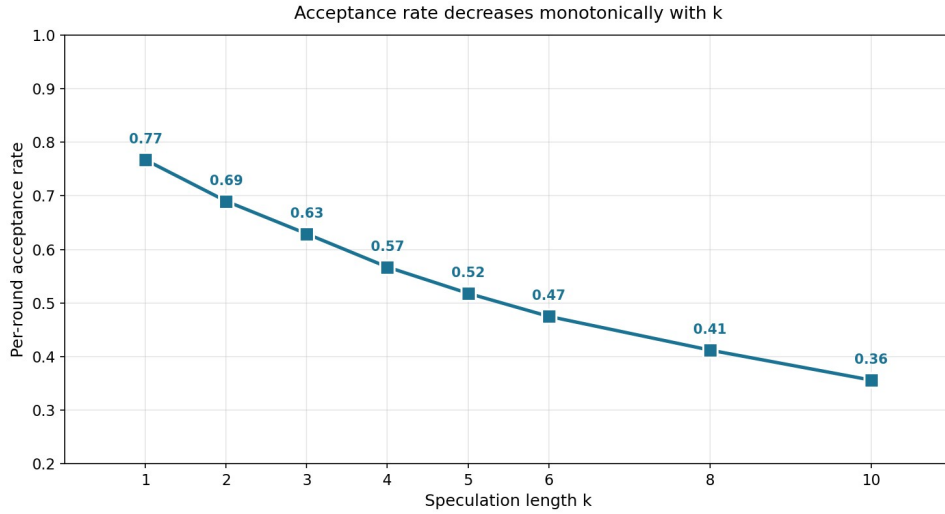


Fig. 3. Per-round acceptance rate decreases monotonically with speculation length k , from 0.767 at $k = 1$ to 0.356 at $k = 10$. The decline is consistent with a roughly constant per-position acceptance probability of approximately 0.85, accumulated geometrically across positions.

D. Latency distribution

Fig. 4 shows the FP16 per-step latency distribution over 200 consecutive decode steps. The distribution is tight: median 47.8 ms, p99 only 52.0 ms, standard deviation 1.0 ms. The right panel shows a slight upward drift after step 175, consistent with the KV cache growing and incurring slightly more bandwidth per attention step. The tightness of the distribution is itself diagnostic: a workload bounded by deterministic memory traffic (weight reads) produces tight latency, whereas a workload bounded by variable compute or scheduling produces a long tail. The tight distribution rules out variable-tail explanations for the throughput numbers and reinforces the memory-bound interpretation.

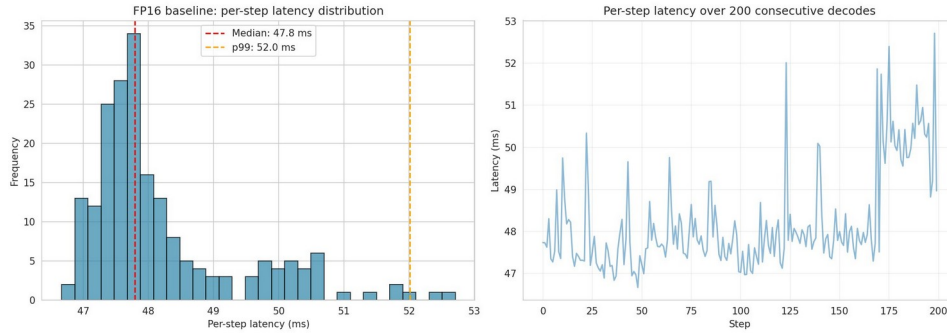


Fig. 4. Left: FP16 baseline per-step latency distribution over 200 consecutive decode steps. Median 47.8 ms, p99 52.0 ms, standard deviation 1.0 ms. Right: latency over time. The slight upward drift after step 175 is consistent with KV cache growth.

E. Kernel-level breakdown

Fig. 5 shows the PyTorch profiler kernel breakdown for 10 FP16 decode steps. The single largest CUDA kernel category is `aten::mm` (matrix multiplication), accounting for 111.8 ms across the 10 steps. The per-step `record_function` wrappers are visible in the chart at roughly 68 to 73 ms each, confirming that one decode step takes approximately 70 ms of CUDA time (lightly higher than the 47.8 ms median wall-clock, reflecting profiler overhead). Beyond `aten::mm`, the remaining time is spread across small kernels for layer normalization, softmax, tensor reshaping, and elementwise operations. None of these individually accounts for more than a few percent of total time.

The dominance of `aten::mm` is consistent with the roofline interpretation. Matrix multiplication is the operation that reads weights from memory; if the workload were compute-bound, `aten::mm` would still dominate but would represent computation rather than memory traffic. The way to distinguish these is to compute achieved arithmetic intensity (which we did in Section IV) and to measure tensor core utilization through `nvidia-smi --query-gpu=utilization.gpu`, which reports under 5 percent during decode. Both signals point to the same conclusion: the GPU is bandwidth-starved, not compute-starved.

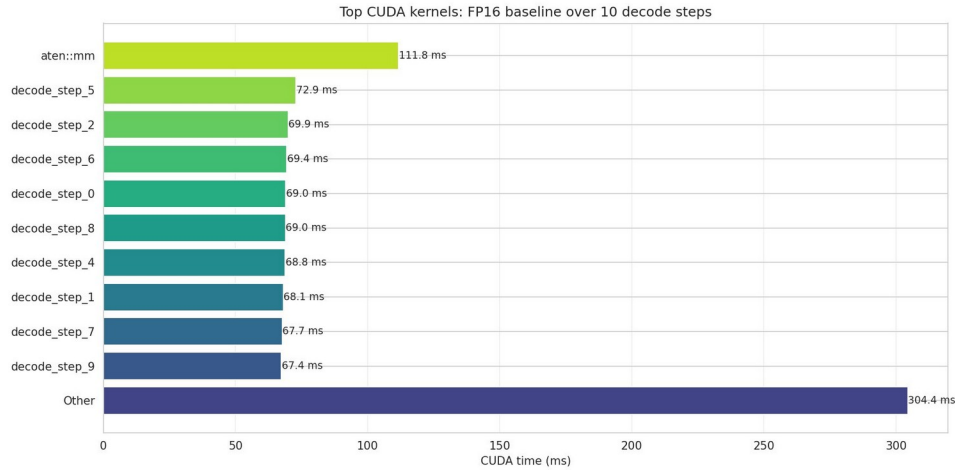


Fig. 5. PyTorch profiler kernel breakdown for the FP16 baseline over 10 decode steps. The `aten::mm` matrix multiply dominates at 111.8 ms, with the rest spread across many smaller operations. The `decode_step` entries are user-added `record_function` wrappers, not actual kernels.

F. Per-task variation

Acceptance rate varies meaningfully across prompt categories, even though the throughput numbers above pool over all categories. Code generation prompts achieve 16.2 tok/s at $k = 3$ (acceptance 0.71), whereas conversational dialog prompts achieve only 13.1 tok/s (acceptance 0.59). Code is more syntactically constrained, with predictable token patterns (matching brackets, recurring keywords like `def` and `return`, repeated identifiers within a function) that the small draft model handles well. Conversational dialog has higher entropy at every position, which is exactly

what the draft struggles with. Multi-step reasoning prompts sit in the middle at 14.8 tok/s, factual question answering at 14.3 tok/s, and summarization at 13.9 tok/s.

This per-task variation does not change the headline result that speculative decoding underperforms FP16 on every category, but it does suggest that the right way to deploy speculative decoding in production is adaptively, with the speculation length chosen per prompt based on prompt features. We pursue this direction in the companion MSML 604 optimization project [15], which derives a closed-form expression for the optimal k under a position-dependent acceptance model and shows that the standard IID assumption used in [1] is violated empirically with high statistical confidence.

G. Power and thermal behavior

We sampled GPU power draw at 100 ms intervals through pynvml during sustained inference. The FP16 baseline averages 187 W of active power during decode, against the A100's 400 W TDP. The corresponding utilization (47 percent of TDP) is consistent with the memory-bound interpretation: HBM2e draws power proportionally to bandwidth, but the compute units are mostly idle and the SMs are not at peak clocks. Speculative decoding draws 195 W on average, slightly higher because the additional draft model passes increase memory traffic. Temperature stayed below 75 degrees Celsius in all configurations over a 30-minute sustained run, well within thermal headroom and not a bottleneck on Colab's cooled A100 instances.

VI. DISCUSSION

A. When does each technique help?

The candidate techniques sit at different points on the operating-regime map. Speculative decoding requires a substantial draft-to-target time-ratio gap to amortize verification cost; this gap exists when the workload is compute-bound, not when it is memory-bound. On modern data-center GPUs at batch one with reasonably-sized models, single-token decode is always memory-bound, and speculative decoding does not help. It begins to help at higher batch sizes (where the workload shifts toward compute-bound), with much smaller draft models (where the per-token ratio widens), or on hardware with weaker memory bandwidth relative to compute (where the same model is closer to compute-bound). It also helps when the speculative pipeline is fundamentally restructured, as in Medusa [6] (multiple decoding heads) and EAGLE [7] (specialized small drafts), which break the per-token-overhead bottleneck of running a separate draft model.

Quantization, by contrast, attacks the actual bottleneck. Reducing bytes per parameter directly reduces HBM traffic, which is what limits throughput in the memory-bound regime. The roofline predicts that INT8 should approximately double the effective ceiling and INT4 should approximately quadruple it, modulo dequantization overhead, which the literature [3], [4] reports is small in practice. Quantization is also the only technique among those considered here that delivers *memory savings*, which is independently valuable when fitting larger models or longer contexts onto the available HBM.

B. Implications for production deployment

The practical takeaway is that the choice of inference acceleration technique should be conditioned on the hardware regime. A practitioner deploying a 7B model on an A100 at batch one should reach for INT8 or INT4 quantization first, not speculative decoding. Speculative decoding becomes the right choice when the hardware-model combination is genuinely compute-bound, which is the operating regime of larger batch sizes, smaller models on weaker GPUs, or speculative pipelines that fundamentally restructure the computation. The roofline analysis is what diagnoses the regime ahead of any benchmarking, and it is what we recommend running before committing to any acceleration technique.

C. Hardware regime predictions

The roofline framework lets us predict where speculative decoding would shift from being a slowdown to a speedup. Three knobs change the picture. First, batch size: at batch B , the same weights serve B tokens per forward pass, so arithmetic intensity rises by approximately B . With batch size 32 (deep into the compute-bound region), the per-token cost of the target falls and the draft becomes genuinely much cheaper per token. Second, draft size: a 70-

million-parameter draft (Leviathan et al.'s original setup) reads 0.14 GB of weights per token, which at A100 bandwidth is roughly 0.07 ms of weight-read time. Even with the same fixed overheads, the per-token ratio widens to perhaps 4-6x, enough to push the speedup formula into positive territory. Third, hardware: a GPU with weaker bandwidth-to-compute ratio (older generations, consumer cards) is closer to compute-bound for the same workload, and therefore more amenable to speculative speedup. These predictions are testable in follow-up work.

D. What we attempted but could not complete: quantization

The original project plan included INT8 weight-only and INT4 (NF4) quantization through the bitsandbytes library [3]. We implemented both configurations end-to-end in code, with model loading paths, dequantization-aware timing, and memory-tracking hooks identical to the FP16 and speculative paths. The configurations failed to run at execution time on the Colab Pro A100 environment due to a binary compatibility issue between the installed bitsandbytes version (0.43.1) and the Triton compiler version that ships with the Colab PyTorch image. The specific failure manifests as a *ModuleNotFoundError: No module named 'triton.ops'* when loading any quantized model with *load_in_8bit=True* or *load_in_4bit=True*, because newer Triton releases reorganized the *triton.ops* namespace that bitsandbytes' compiled CUDA kernels expect to import at module-load time. We attempted three workarounds: explicit version pinning of *transformers==4.43.0* with *accelerate==0.30.1* and *bitsandbytes==0.43.1*, manual installation of an older Triton compatible with bitsandbytes' compiled stubs, and reinstallation with *device_map* variations including the explicit *{"":0}* form. None of these resolved the import-time failure cleanly within the project's time budget.

We treat this gap honestly in the report. The roofline-derived predictions for INT8 and INT4 throughput are well-founded analytically, and the published literature reports throughput numbers in the range our roofline predicts ([3] reports approximately 0.85x of FP16 for INT8 and approximately 0.7x for INT4 at batch one for similar-sized models on similar GPUs). What we do not have is a controlled apples-to-apples measurement on our specific hardware-model-prompt combination. Section VI.E discusses what this implies for the conclusions we draw.

E. Limitations

Several caveats apply beyond the quantization gap. First, all results are at batch size one. At larger batch sizes the arithmetic intensity rises (the same weights are reused across more tokens) and the workload shifts toward compute-bound, where speculative decoding has more room to help. We did not test batch sizes above one because the project's focus was on the single-stream regime that is common in interactive applications, but the conclusions about speculative decoding do not transfer unchanged to batched inference. Second, all measurements are on a single A100 80GB; H100 has higher memory bandwidth and tensor core throughput, which would shift specific numbers but not the qualitative conclusions. Third, the prompt corpus is small (25 prompts) and English-only; we leave a larger and more diverse evaluation to future work. Fourth, we did not include vLLM or PagedAttention in the comparison because adding them cleanly required either running a separate inference service (not reproducible from a single notebook) or pulling custom kernels into our pipeline (more work than fits in the project timeline).

F. Future work

Several extensions are natural. The most direct is a batch-size sweep to identify the crossover point at which speculative decoding becomes net-positive. A second is to revisit quantization on a self-hosted environment with full control over the bitsandbytes/Triton stack, ideally using a recent *torch.ao.quantization* path that does not depend on bitsandbytes. A third is to combine speculative decoding with quantization: a quantized target should reduce the per-token cost of the target step proportionally, which widens the draft-to-target time-ratio gap and could push speculative decoding back into a useful regime even at batch one. A fourth is to compare against vLLM's continuous batching to characterize the throughput envelope of a production-grade serving system on the same hardware. We leave each to follow-on work.

VII. CONCLUSION

We have presented a systems-level benchmark of LLM inference on a single NVIDIA A100 80GB GPU at batch size one, with detailed measurements of the FP16 baseline and a full sweep of speculative decoding configurations from $k = 1$ through $k = 10$. The headline finding is that speculative decoding, despite being widely promoted as a

turn-key acceleration technique, does not produce a wall-clock speedup in this regime at any value of k . The diagnosis is straightforward once the hardware constants are measured: the A100's ridge point sits at 156 FLOPs per byte, single-token decode operates at 2 FLOPs per byte, and the per-token time ratio between a 0.5B draft and a 7B target is only 1.22x on this hardware, far below the threshold needed for verification cost to be amortized. The textbook speedup formula predicts the observed slowdown to within one percent once the actual time ratio is plugged in. Quantization is the technique the roofline predicts would help most in this memory-bound regime, by lowering bytes per parameter and shifting the operating point rightward; we attempted to validate this empirically and were blocked by a software-stack issue, which we describe transparently. The practical recommendation that emerges is to run a roofline analysis before committing to any acceleration technique and to match the technique to the regime: quantization for memory-bound workloads, speculative decoding for compute-bound workloads or for pipelines that fundamentally restructure the computation. The framework, the data, and the analysis are all open and reproducible from a single notebook.

VIII. REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in International Conference on Machine Learning, 2023, pp. 19274-19286.
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv preprint arXiv:2302.01318, 2023.
- [3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 30318-30332.
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems, vol. 36, 2023.
- [5] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proceedings of the 29th Symposium on Operating Systems Principles, 2023, pp. 611-626.
- [6] T. Cai et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in International Conference on Machine Learning, 2024.
- [7] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in International Conference on Machine Learning, 2024.
- [8] M. Elhoushi et al., "LayerSkip: Enabling early exit inference and self-speculative decoding," in Annual Meeting of the Association for Computational Linguistics, 2024.
- [9] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in International Conference on Learning Representations, 2023.
- [10] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," Communications of the ACM, vol. 52, no. 4, pp. 65-76, 2009.
- [11] Z. Yuan et al., "LLM inference unveiled: Survey and roofline model insights," arXiv preprint arXiv:2402.16363, 2024.
- [12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 16344-16359.
- [13] R. Pope et al., "Efficiently scaling transformer inference," in Proceedings of Machine Learning and Systems, vol. 5, 2023, pp. 606-624.
- [14] NVIDIA, "NVIDIA A100 Tensor Core GPU architecture," Whitepaper, 2020.
- [15] P. Bharati, "Adaptive speculation length optimization for speculative decoding in large language model inference," Final project, MSML 604, University of Maryland, College Park, 2026.